

CS 40700 Design Document

Product Name: Yarrow

Team Number: 13

Team:

Aagam Bapna - abapna@purdue.edu

Cheng-En Lee - lee4649@purdue.edu

Anthony McCrovitz - amccrov@purdue.edu

Daniel Benes-Magana - dbenesma@purdue.edu

Jaime Xu - xu1545@purdue.edu

CS 40700 Design Document	1
1. Purpose	2
2. Design Outline	2
2.1 Proposed System Architecture	2
2.2 System Components	2
2.3 Component Interactions and UML Diagrams	3
2.4 Frontend State Diagram	3
3. Design Issues	4
3.1 Functional Issues	4
3.2 Non-Functional Issues	4
4. Design Details	6
4.1 Class-Level Design	6
Database Schema Design	6
UML Class Diagram	
This UML class diagram shows the main classes in Yarrow and the relationships between them.	7
4.2 Class Descriptions and Interactions	7
4.3 Sequence Diagrams	8
4.4 Activity/State Diagrams	11
4.5 UI Mockups	11

1. Purpose

Scanned documents often contain valuable information that is difficult to extract and search reliably because tables, figures, complex layouts, and page breaks complicate automated parsing. Existing document-processing workflows often separate document extraction, large-scale processing, cross-document search, and source verification into different steps or systems.

Yarrow would be a web-based document parsing platform designed that integrates all of these features together. By allowing users to upload batches of scanned PDFs and images, the system would convert complex content into structured, searchable data. It also would ensure traceability by linking extracted information back to its source page and region, offering a unified, efficient workflow for processing, searching, and verifying large document collections.

2. Design Outline

2.1 Proposed System Architecture

Our system would adopt a client-server architecture. The frontend would make requests to the main backend and receive results. Then, the backend would delegate document parsing tasks to backend worker nodes, the worker nodes would then send inference requests to the nginx vllm load balancer, which would distribute these requests to underlying vllm nodes. To communicate between components, REST would be used.

2.2 System Components

Frontend: The frontend provides the web interface through which users interact with Yarrow. It will support document uploads, document management, processing status, search, and viewing processed documents alongside their extracted data and bounding boxes.

Backend: The backend provides the API used by the frontend and handles application logic such as authentication, document management, processing requests, and communication with the system's storage and processing components.

Document Storage: Uploaded PDFs and images are stored separately from the application's structured data. The current proposed design uses AWS S3 for storing the original uploaded documents.

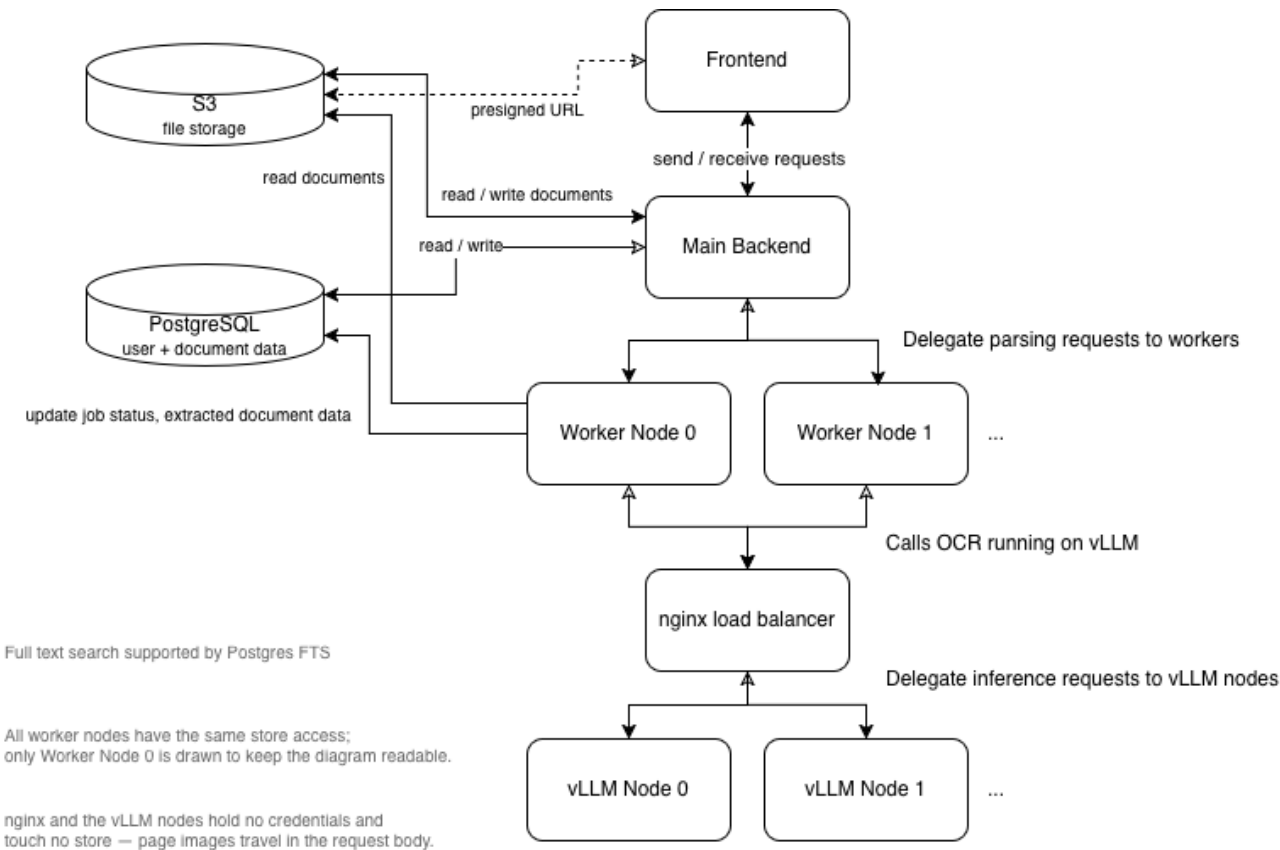
PostgreSQL Database: PostgreSQL stores structured application data such as user information, document metadata, processing status, and extracted document information. ParadeDB would be used on top of PostgreSQL database to provide text search capabilities, allowing users to retrieve document pages with a search query.

Document Processing Workers: Background processing workers handle computationally intensive document-processing tasks. These workers process uploaded pages, extract text and layout information, communicate with inference services when necessary, and store processing results.

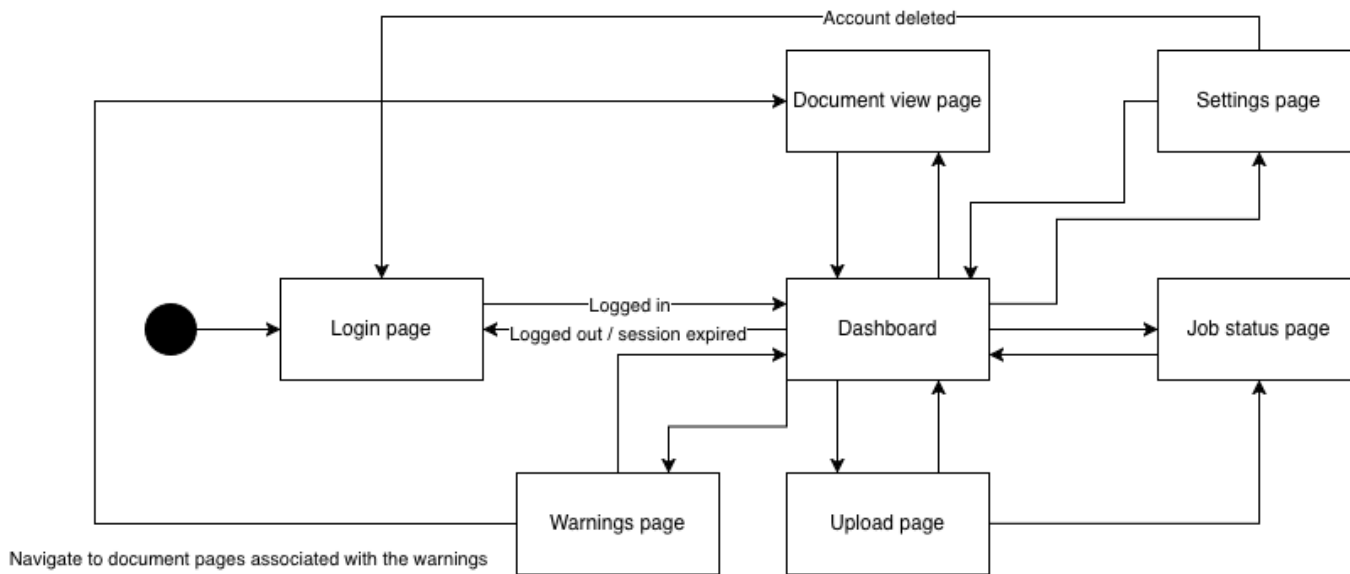
Inference Services: Inference services run the OCR, vision-language, and other machine-learning models used during document processing. The specific inference serving technology is still being evaluated.

Task Queue: The task queue coordinates background document-processing jobs between the backend and processing workers. It allows long-running processing to occur asynchronously and supports features such as job progress, retries, and distributed processing.

2.3 Component Interactions and UML Diagrams



2.4 Frontend State Diagram



The frontend state diagram shows the major pages and navigation paths available to a user in Yarrow. After successfully logging in, the user enters the dashboard, which serves as the primary navigation point for uploading documents, viewing processing jobs and warnings, opening processed documents, and accessing account settings. Users can move between these workflows and return to the dashboard, while logging out or allowing the session to expire returns the user to the login page.

3. Design Issues

3.1 Functional Issues

1. How does a user log in to Yarrow?

Option 1: Use custom username and password authentication

Option 2: Use third-party authentication providers

Why Option 1: This allows full control over the user database and authentication logic, with no external dependencies.

2. Should user account creation be rate-limited? If so, how?

Option 1: No

Option 2: Yes. Users have to verify their email. No two accounts can have the same email.

Option 3: Yes. Rate-limit by IP

Why Option 2: If user account creation is not rate-limited, there is a risk that an adversary might create large amounts of accounts and still starve the storage resource. Merely rate-limiting by IP is not enough, as VPNs can change the user's IP without much cost. Option 2 is a more effective mitigation since it requires an account to be uniquely tied to an email account, which is significantly more costly to an attacker.

3. Should each user have an upload quota limit?

Option 1: No. Users can upload any amount of files at any rate as they want.

Option 2: Yes, only limit the rate of file upload (Eg. 100 / day) to block out automated attacks.

Option 3: Yes, limit the total amount of files they can upload.

Option 4: Yes, limit the total number of bytes they can upload.

Why Option 4: Leaving users uploading any amount of files at any rate can lead to security risks that might allow attackers to starve storage resources. Limiting file upload rate only partially mitigates the risk, but storage could still easily run out since the user upload data is unbounded. Limiting the total amount of files is ineffective since a file may contain numerous pages. Limiting the total amount of bytes users can upload is the only method to guarantee that the user upload data is bounded.

3.2 Non-Functional Issues

1. What backend framework should Yarrow use?

Option 1: Express.js

Option 2: FastAPI

Option 3: Flask

Option 4: Django

Why Option 2: FastAPI is a Python backend framework. Since Python has great support on AI / ML tasks, we believe that adopting a Python backend framework would make development easier and the resulting codebase cleaner. FastAPI is an ASGI-based backend framework that can handle concurrent tasks more efficiently than WSGI-based backend frameworks like Flask, therefore more suited for the goal of handling document parsing requests concurrently. Django is an highly-opinionated framework, which means it includes built in ORM, admin dashboard, frontend, etc. Most of the built in features Django provides are unnecessary.

2. What database should the backend use?

Option 1: PostgreSQL

Option 2: MySQL

Option 3: MongoDB

Why Option 1: As a robust relational database, PostgreSQL is highly efficient for managing structured data. It provides excellent support for complex data structures and includes built-in full-text search capabilities, making it a comprehensive and performant choice for our backend.

3. Do we need a separate elasticsearch database for full text search?

Option 1: Yes, use elasticsearch

Option 2: No, use postgres FTS instead

Option 2.5: use postgres+paradedb

Why Option 2.5: This option is simpler in architecture compared to Option 1. No need to set up an extra deployment for elastic search. This also rules out the possibility where text stored in two databases disagree with each other. paradedb is a search engine built on pg_search extension that offers full text search as well as advanced features such as relevance scoring, vector search, and hybrid search. This provides us great flexibility on extending the search capabilities of Yarrow.

4. How should processing failures be handled?

Option 1: Fail the entire document.

Option 2: Retry only failed pages.

Why Option 2: Retrying individual pages avoids reprocessing the entire document and makes the system more reliable.

5. How should documents be processed?

Option 1: Process each document on one worker.

Option 2: Process pages across multiple workers.

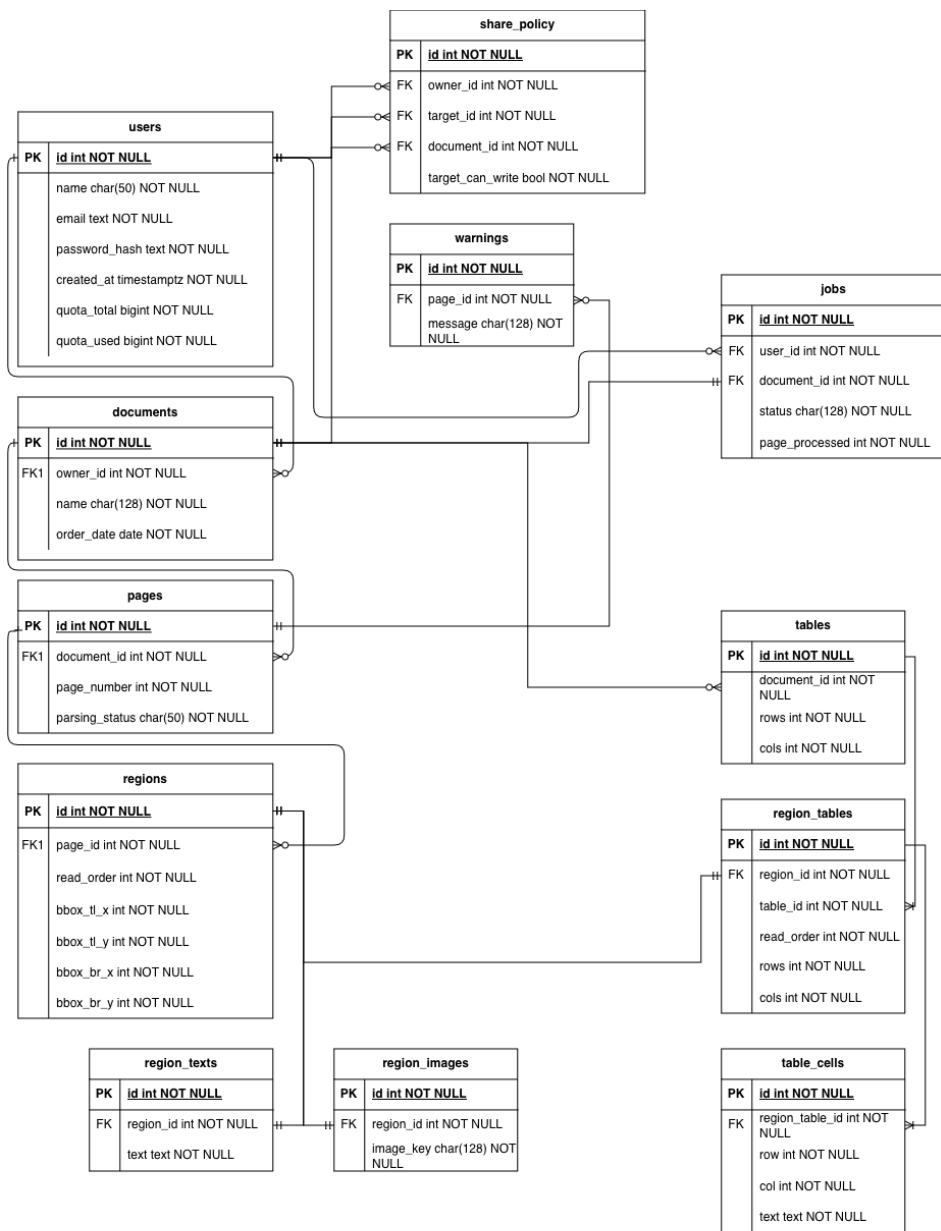
Why Option 2: Pages can be processed independently, allowing large documents to be processed faster.

4. Design Details

4.1 Class-Level Design

Database Schema Design

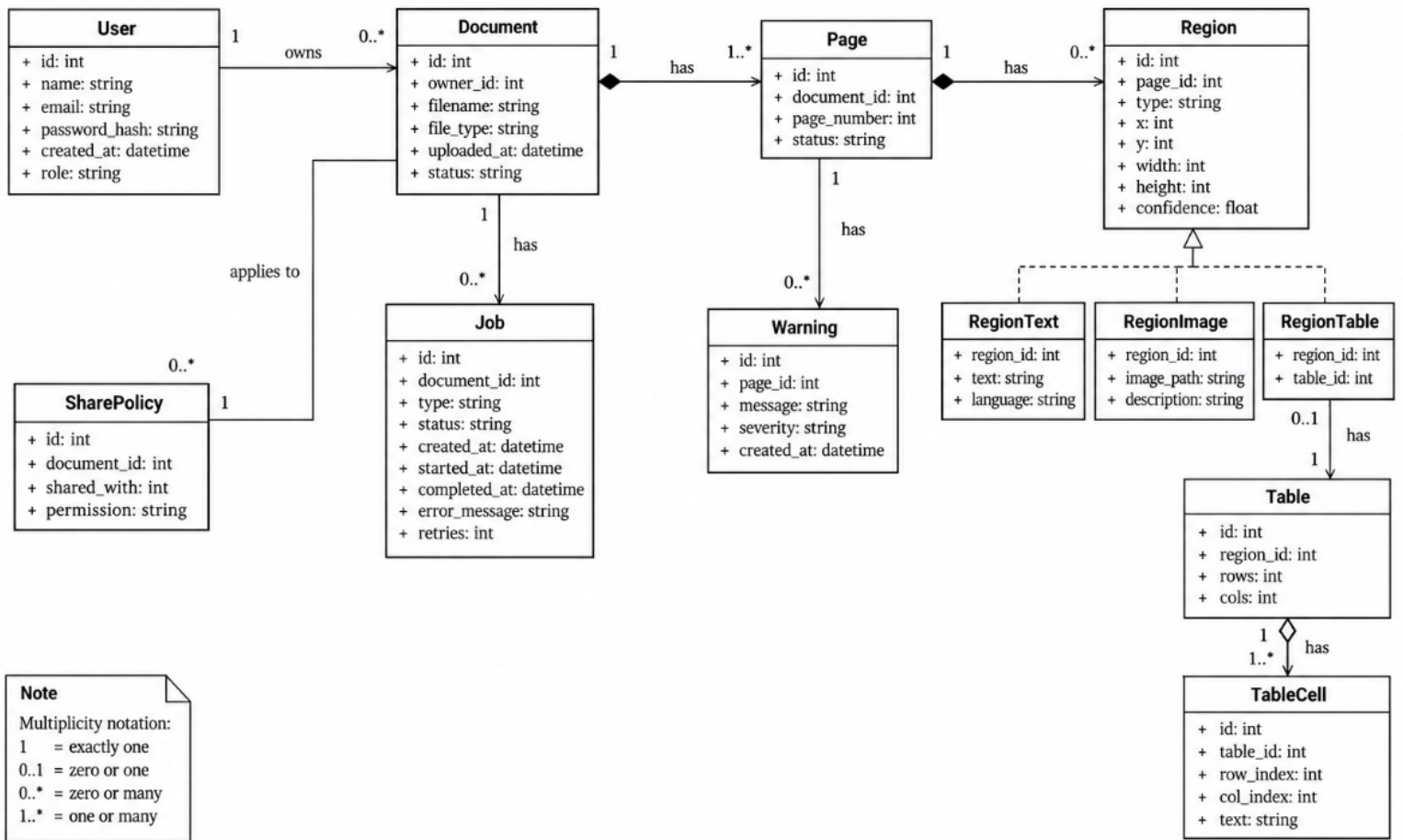
PostgreSQL database we use would contain the following data structures:



nodes	
PK	id int NOT NULL
	node_type text NOT NULL
	last_heartbeat timestampz NOT NULL
	active_jobs int NOT NULL
	model_version text
	details jsonb

UML Class Diagram

This UML class diagram shows the main classes in Yarrow and the relationships between them.



4.2 Class Descriptions and Interactions

The data classes in Yarrow correspond to the entities shown in the database schema in Section 4.1. Although these entities are represented as PostgreSQL tables in the current design, they define the core data objects used to represent users, uploaded documents, extracted content, processing jobs, warnings, sharing permissions, and processing nodes.

User: Represents a registered Yarrow user. A user stores account information including a name, email address, password hash, and account creation time. Users can own documents, initiate processing jobs, and participate in document-sharing relationships.

Document: Represents a document uploaded to Yarrow. Each document is associated with an owner and stores identifying document metadata. A document can contain multiple pages, processing jobs, extracted tables, and sharing policies.

Page: Represents an individual page within a document. Each page belongs to a document and stores its page number and parsing status. Pages can contain multiple extracted regions and processing warnings.

Region: Represents an extracted area within a page. Each region stores its reading order and bounding-box coordinates, allowing extracted content to be linked back to its exact location in the original document. A region can be associated with extracted text, an extracted image, or part of an extracted table.

RegionText: Stores text extracted from a specific region and maintains a reference to the corresponding Region.

RegionImage: Stores a reference to image data extracted from a specific region using an image key while preserving the connection to the corresponding Region.

Table: Represents a logical table extracted from a document and stores its overall row and column dimensions.

RegionTable: Associates a Region with a Table and stores region-level table information such as reading order and row and column dimensions. This allows a logical table to remain connected to the document region from which it was extracted.

TableCell: Represents an individual extracted table cell. Each cell belongs to a RegionTable and stores its row position, column position, and extracted text.

Job: Represents a document-processing job initiated by a user for a document. A job tracks its current status and the number of pages processed.

Warning: Represents a warning associated with a particular page. Warnings allow processing or document-quality issues to be recorded and shown to the user for the page on which they occurred.

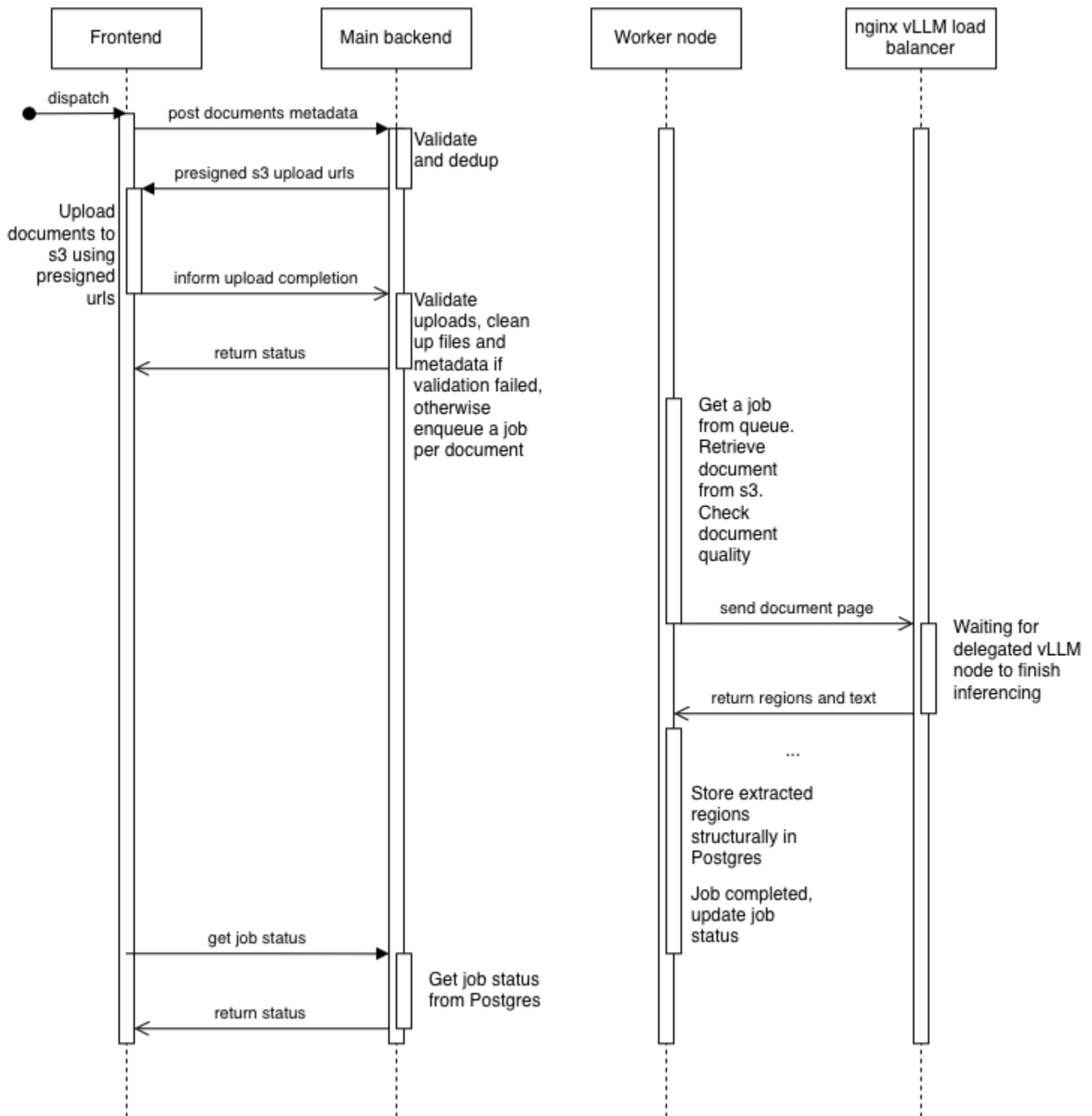
SharePolicy: Represents document-sharing permissions between users. It records the document owner, the target user, the document being shared, and whether the target user has write access.

Node: Represents a processing or inference node available to Yarrow. It stores information including the node type, last heartbeat, active job count, model version, and additional node-specific details.

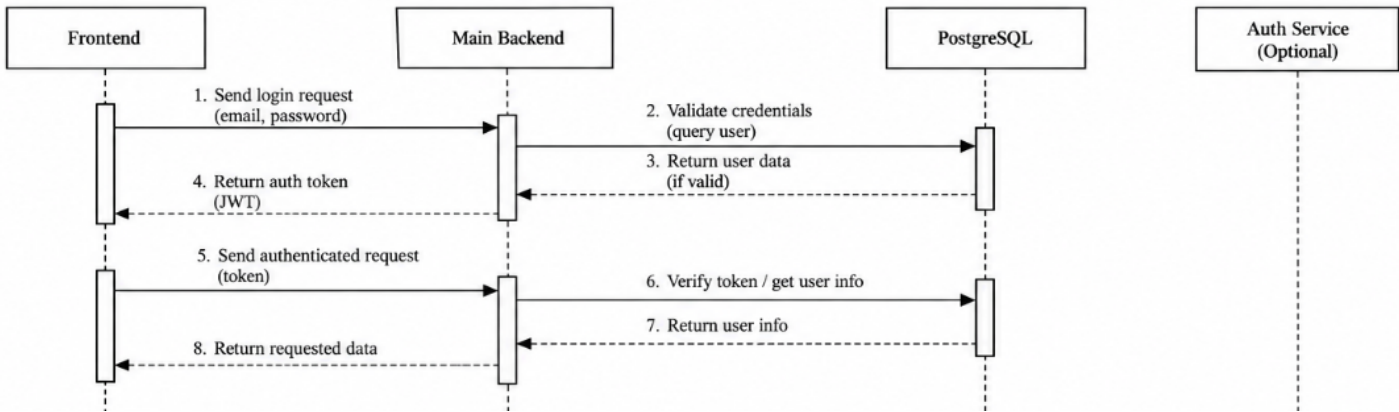
Class Interactions: A User owns Documents and can initiate Jobs that process those documents. Each Document is divided into Pages, and each Page can contain multiple Regions and Warnings. Regions preserve the source location of extracted content and connect to RegionText, RegionImage, or RegionTable records. RegionTable connects document regions to logical Tables, while TableCell stores the individual values extracted from those tables. Jobs track document-processing progress, SharePolicy records control access between users, and Node records represent the processing resources available to the system.

4.3 Sequence Diagrams

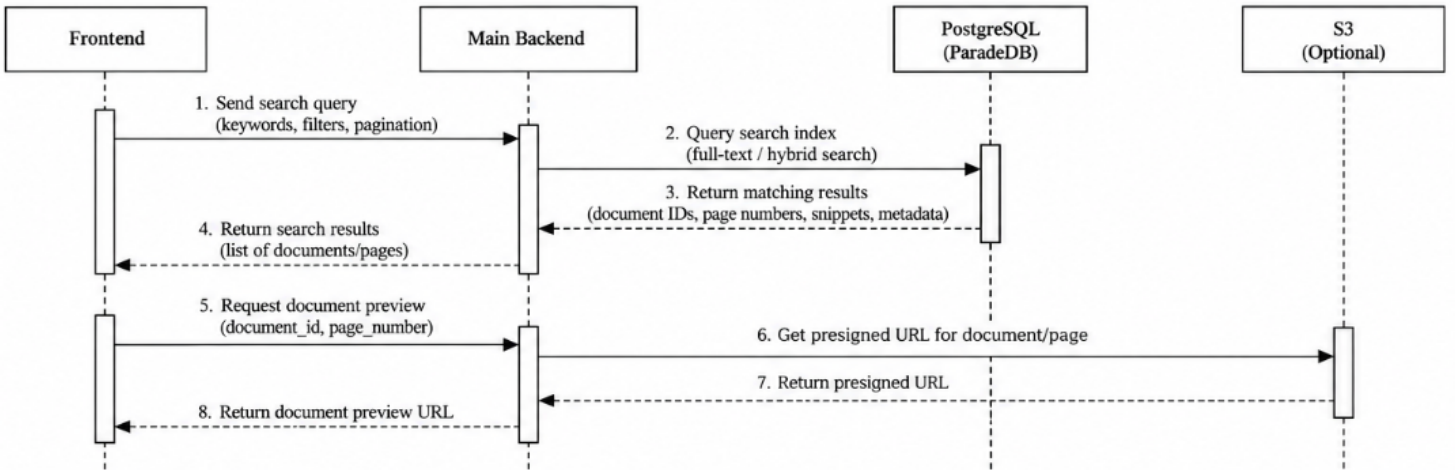
When a user uploads documents, the system undergoes the following actions in a sequence:



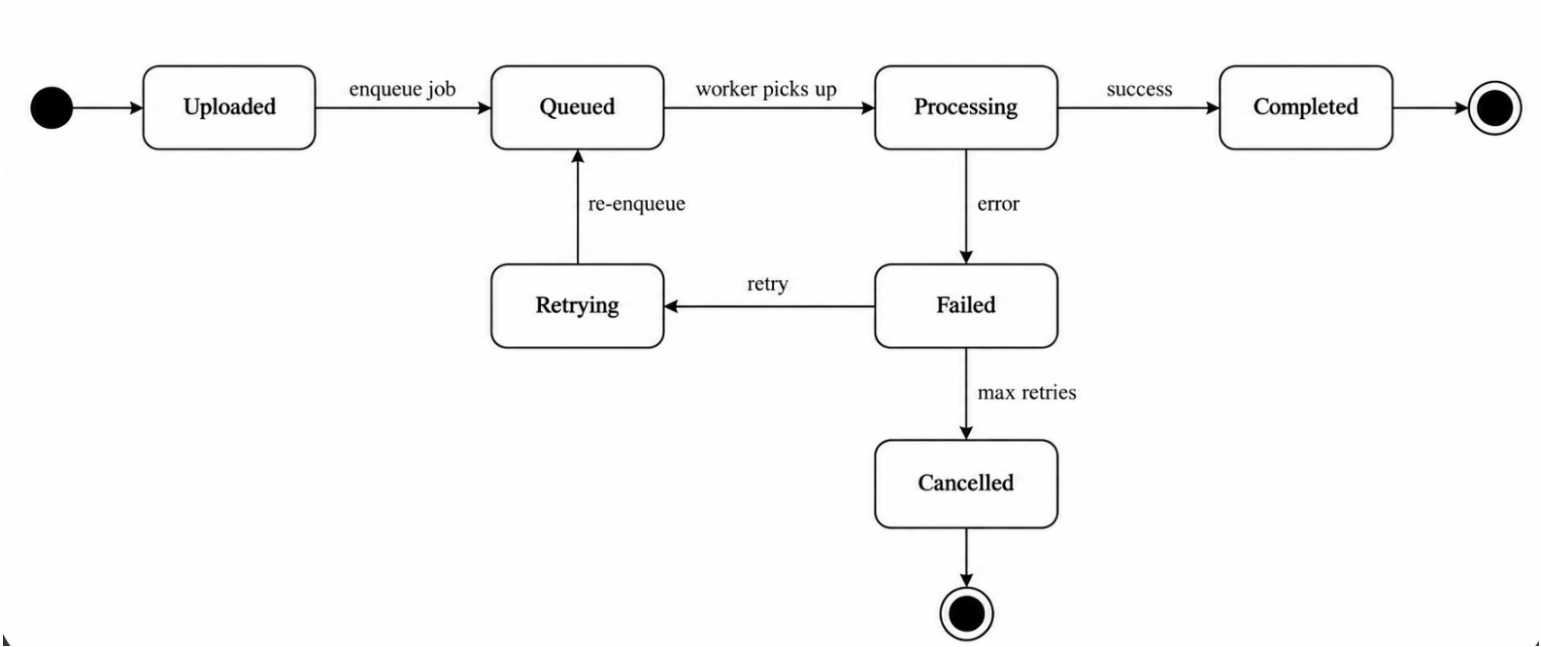
Sequence Diagram: User Login



Sequence Diagram: Search Documents



4.4 Activity/State Diagrams



4.5 UI Mockups

The following wireframes represent preliminary UI concepts for Yarrow’s core user workflows. These mockups are intended to communicate the expected layout and functionality of the system and are not representative of the final visual design. The interface may change as implementation progresses and additional usability feedback is collected.

Dashboard / Document Library

This screen serves as the main workspace for viewing and managing uploaded documents. Users can search their document library, review processing status, open completed documents, and start a new upload.

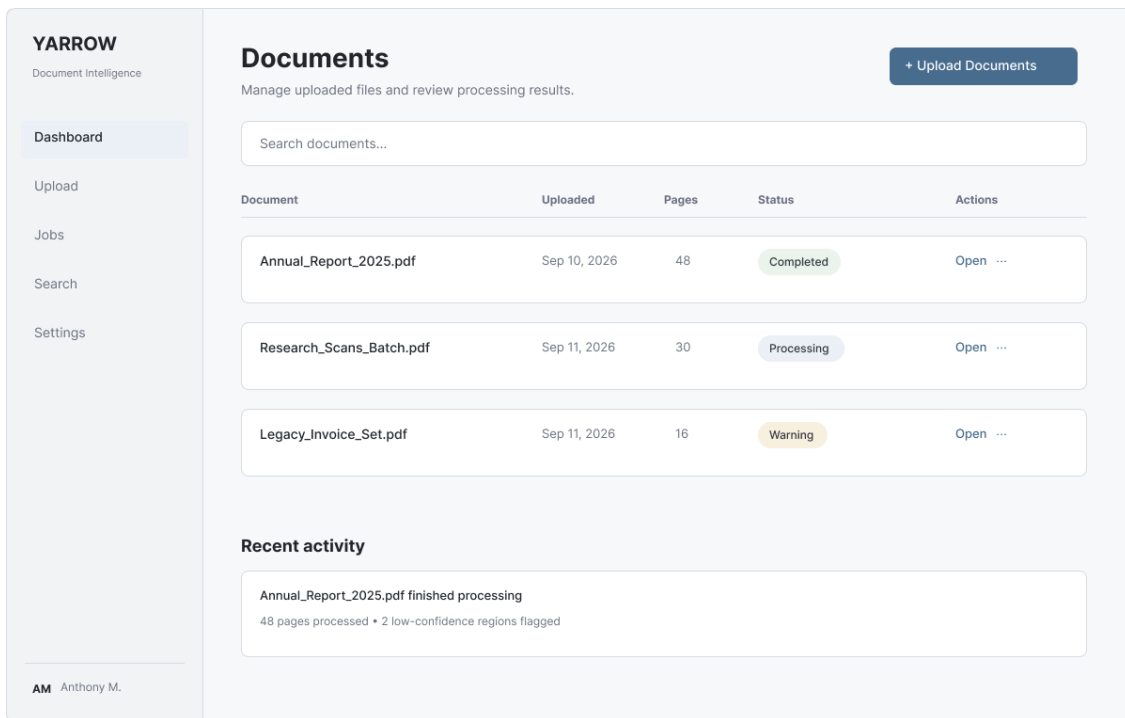


Figure 1: Dashboard / Document Library

Bulk Document Upload

This screen allows users to upload multiple PDFs or images at once. It shows selected files before processing begins and gives the user a clear action to start document processing.

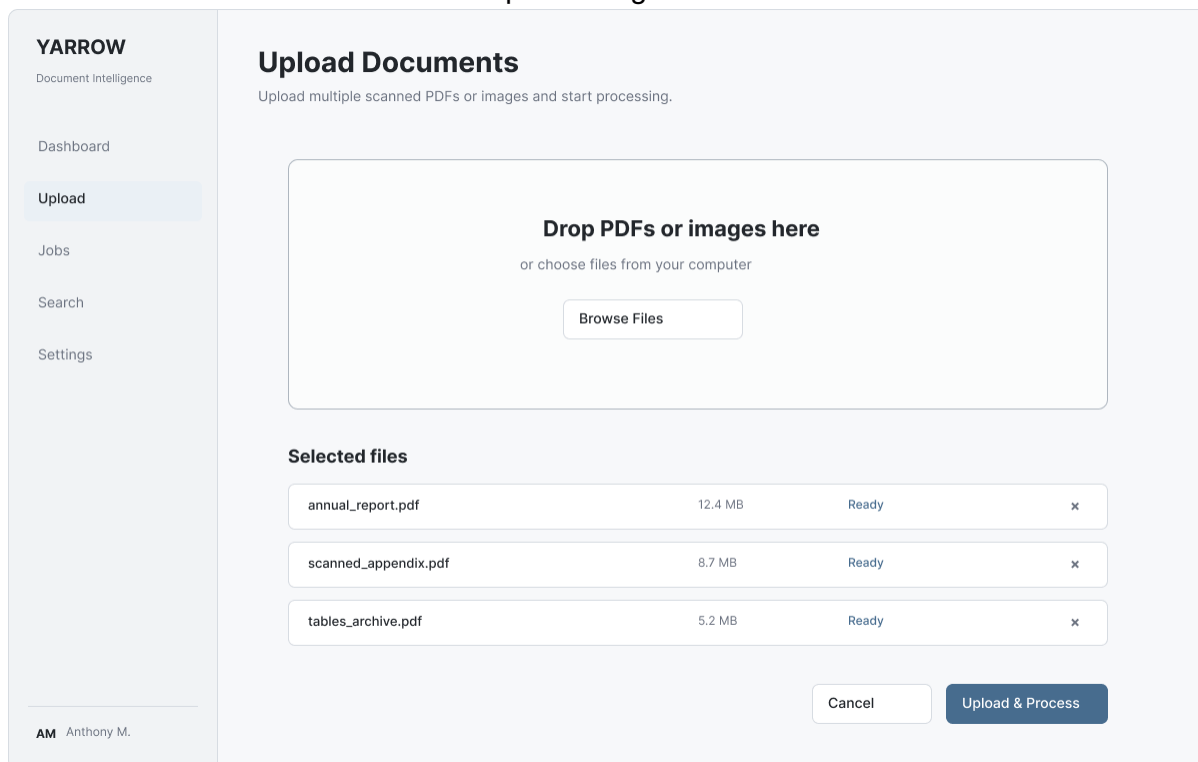


Figure 2: Bulk Document Upload

Processing Job Status

This screen allows users to monitor queued, active, completed, and failed processing jobs. Progress, page counts, warnings, and retry actions make the state of long-running document processing visible to the user.

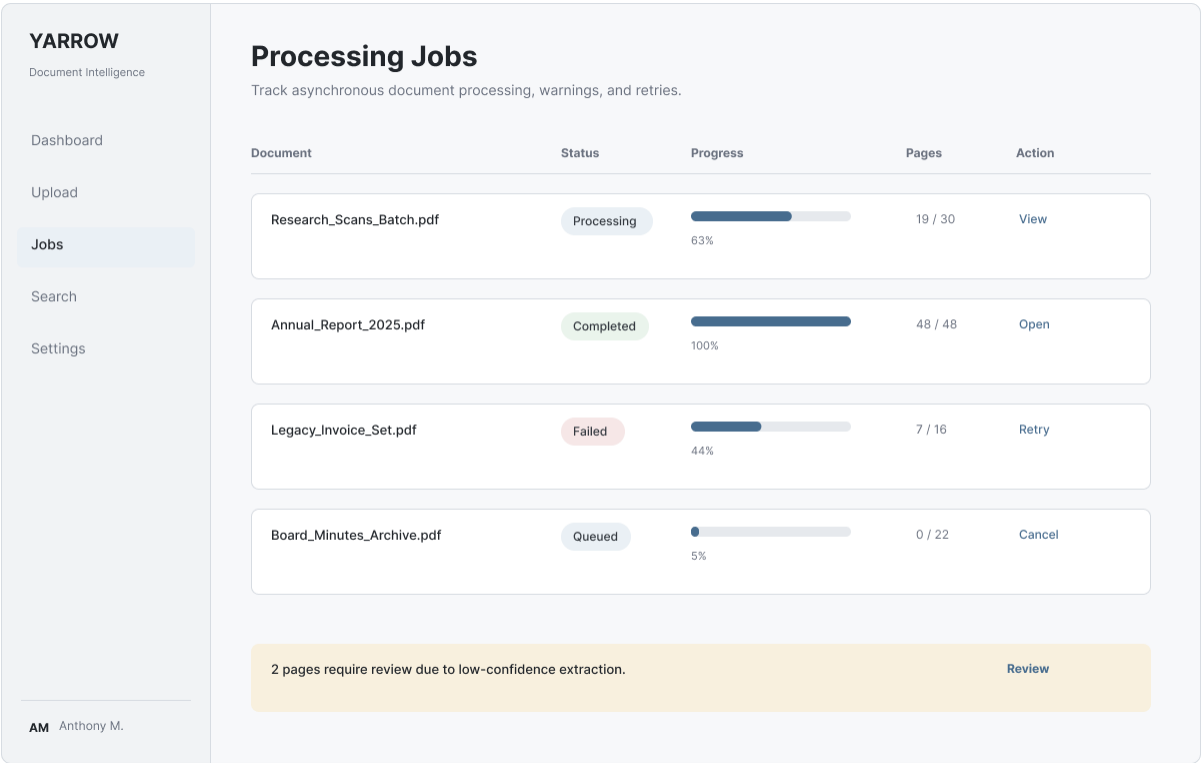


Figure 3: Processing Job Status

Document Viewer

This screen allows users to compare the original document with the structured data extracted by Yarrow. Bounding boxes connect extracted text, tables, and images to their corresponding source regions so users can verify results and review warnings.

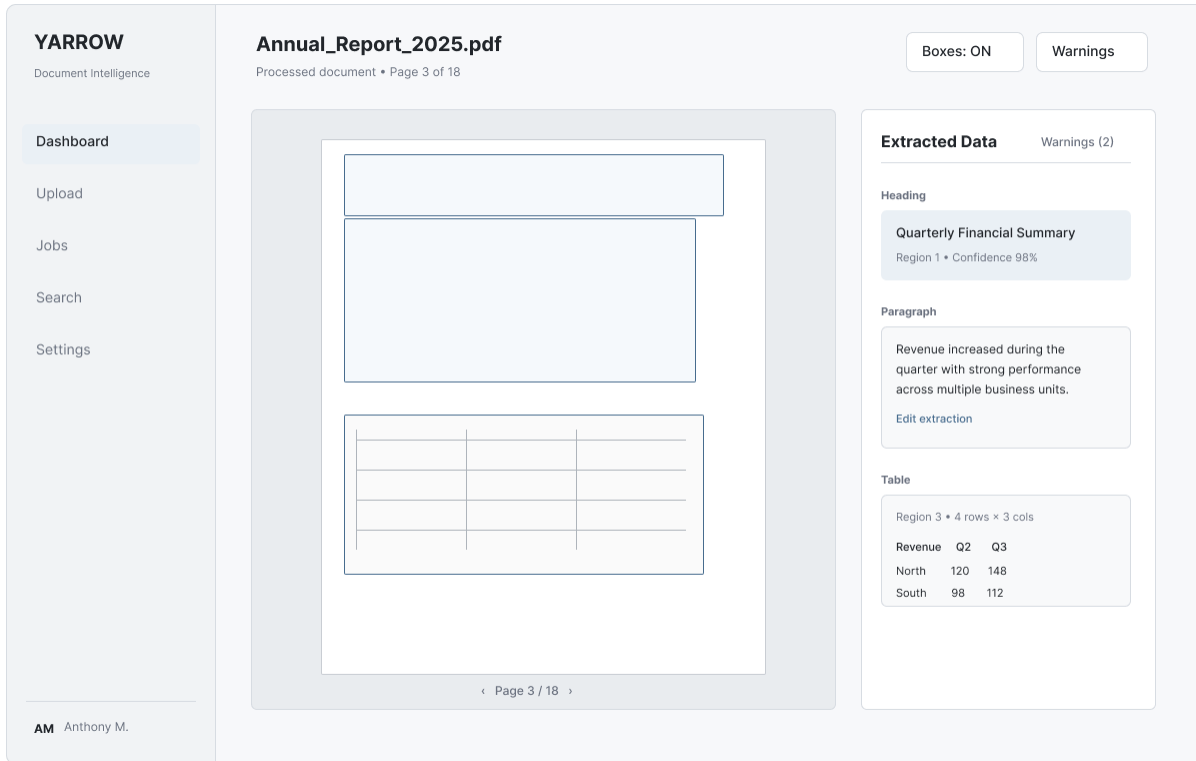


Figure 4: Document Viewer

Search Results

This screen allows users to search extracted information across processed documents. Results include document and page context and provide a direct path back to the matching source location in the document viewer.

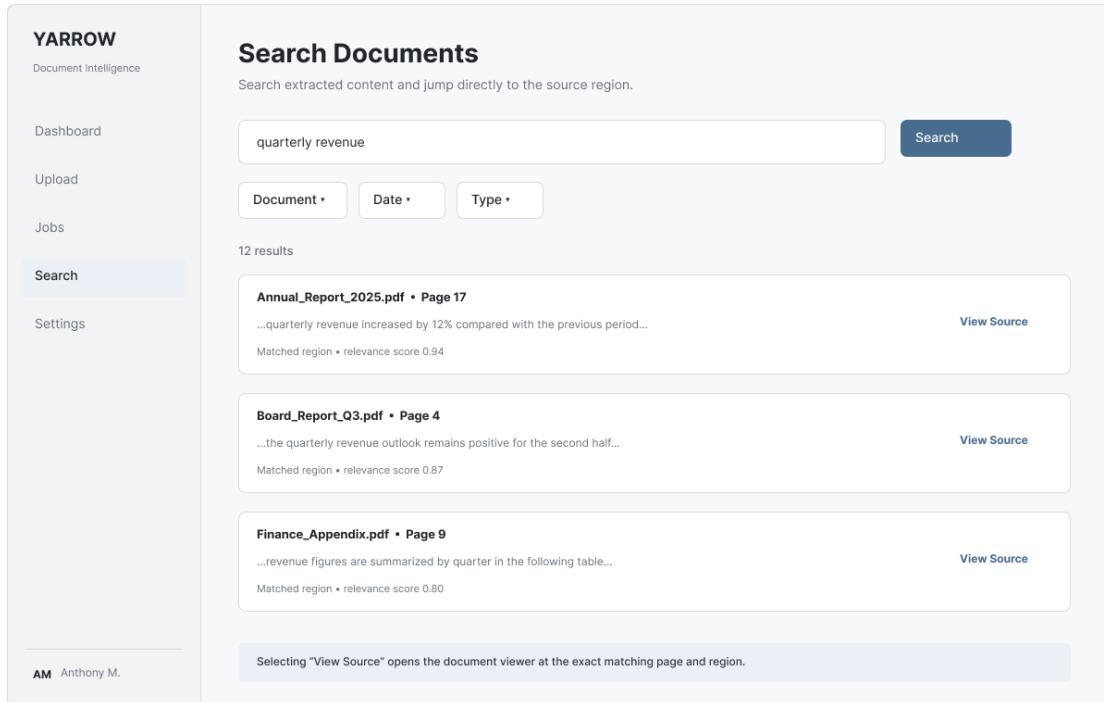


Figure 5: Search Results

Login / Register

This screen represents the entry point for user accounts in Yarrow. Users can sign in with an existing account or begin account creation, with email verification used before access to uploads and document processing.

YARROW

Document Intelligence

Securely process, search, and verify complex documents.

Sign in to Yarrow

Access your document workspace and processing jobs.

Email

Password

[Forgot password?](#)

Sign in

or

Create an account

New users verify their email before accessing uploads.

Preliminary wireframe - final authentication design may change.

Figure 6: Login / Register